

Łukasiewicz Neural Networks Extended: Residual Architectures and Crystallization Strategies for Interpretable Rule Extraction

Carlos Leandro

Departamento de Matemática, Instituto Superior de Engenharia de Lisboa (ISEL),

Instituto Politécnico de Lisboa, Portugal

`carlos.leandro@solidreturn.dev`

Abstract—A feed-forward neural network whose weights are restricted to $\{-1, 0, +1\}$ and whose activation is the truncated identity implements the connectives of Łukasiewicz many-valued logic exactly, neuron by neuron. Once weights are driven to integers — a process called *crystallization* — the network is not a black-box but a logical formula readable directly from its topology. Two obstacles have constrained this approach: the original Levenberg–Marquardt (LM) training fails on high-dimensional problems, and the symbolic guarantees held only for shallow architectures.

This paper resolves both. We prove that *residual skip connections* extend Łukasiewicz networks to arbitrary depth while preserving the symbolic correspondence by construction: merge neurons automatically satisfy the neuron-classification criterion regardless of what the inner layers compute. We then compare three crystallization strategies — corrected LM, straight-through estimation (STE), and proximal regularization — in both flat and residual variants across six UCI benchmarks (10 independent trials each). STE with residual connections achieves the highest mean F_1 on four of six benchmarks and significantly outperforms its flat counterpart on structured datasets. LM with residual connections recovers exact logical formulas on structured problems. Proximal regularization collapses to majority-class predictions on five of six datasets regardless of architecture, a structural failure mode that is theoretically characterized. A comparison with contemporaneous differentiable logic gate networks identifies the complementary strengths of the two approaches. **Keywords:** neuro-symbolic AI; Łukasiewicz logic; interpretable machine learning; symbolic rule extraction; residual networks; weight crystallization; straight-through estimation.

I. INTRODUCTION

The tension between predictive power and interpretability in machine learning has deepened as models have grown larger. A neural network that achieves human-level performance on a medical diagnosis task may be clinically useless if its reasoning cannot be audited. Rudin [1] argues that in high-stakes domains

the correct response is not to add a post-hoc explanation layer on top of a black-box model, but to require that the model itself be interpretable by construction. Post-hoc explanation methods provide approximations — and an approximation to a decision can be systematically wrong in ways that are invisible to the explainer [2].

Symbolic rule extraction from neural networks has pursued this goal since the 1980s [3–5]. The dominant paradigm trains a network, then searches for a logical rule that approximates its input-output behaviour. The approximation step is unavoidable in classical two-valued logic: a neuron computing a weighted sum is not, in general, a Boolean connective. Łukasiewicz logic removes this obstacle. Its connectives — conjunction, implication, negation, and disjunction — are all piecewise linear functions on $[0, 1]$, the same domain that truncated-identity neurons compute over. Castro and Trillas [6] observed that this structural coincidence is not accidental: a neuron with integer weights $\{-1, 0, 1\}$ and truncated identity activation *is* a Łukasiewicz connective, exactly, without approximation. Leandro [7] turned this observation into a complete methodology: train a feed-forward network so that its weights crystallize to integers, then read the resulting formula directly from the network topology.

The programme is mathematically clean and practically motivated, but two limitations have constrained its reach. First, the Levenberg–Marquardt (LM) training algorithm used in the original work crystallizes weights reliably on small, low-dimensional datasets, but suffers from a numerical instability when weights are near zero at initialisation — a regime that Łukasiewicz networks enter naturally — causing

divergence before convergence.¹ We identify and correct this instability; the correction restores LM’s effectiveness on problems where it previously failed. Second, the symbolic correspondence was proven only for networks with a single hidden layer of classifiable neurons [6, 7]; it does not extend trivially to deeper architectures, because training cannot guarantee that every neuron in every additional layer satisfies the classification criterion.

This paper extends the Łukasiewicz neural network (ŁNN) framework along both dimensions. On the theoretical side, we show that *residual connections* — the skip-connection mechanism introduced by He et al. [8] — can be added to ŁNNs while preserving the symbolic correspondence by construction. The key insight is architectural: the neuron that combines the skip path and the inner-layer path — the *merge neuron*, which receives exactly two inputs each with weight +1 — automatically satisfies the neuron-classification criterion regardless of what the inner layers compute. On the empirical side, we compare three crystallization strategies (corrected LM, straight-through estimation [9], and proximal regularization) in both flat and residual architectures, characterizing the conditions under which each strategy succeeds or fails.

a) Contributions.:

- 1) **Residual Łukasiewicz networks (theory).** We prove that merge neurons in a Łukasiewicz residual block satisfy the neuron-classification property (Proposition 2) by construction, extending the symbolic framework to arbitrary depth.
- 2) **Crystallization analysis.** We identify and correct a numerical instability in the original LM training, and characterize three crystallization strategies (LM-corrected, STE, proximal) with their theoretical guarantees and failure modes.
- 3) **Empirical evaluation.** Five training configurations across six UCI benchmarks (10 independent trials each) show that STE with residual connections significantly outperforms flat STE on structured datasets and achieves the best aggregate discriminative quality overall.
- 4) **Formula extraction in practice.** We demonstrate extraction of exact logical formulas from trained networks, including a clinically interpretable rule for cardiac disease diagnosis.

¹The instability arises from the Marquardt-form damping $\mu \text{diag}(J^\top J)$: when weights are small the diagonal entries of $J^\top J$ vanish, making the system ill-conditioned. Section IV-B derives the corrected Levenberg form and quantifies the magnitude of the divergence.

The remainder of the paper is organized as follows. Section II reviews Łukasiewicz logic and the original ŁNN framework. Section III introduces residual ŁNNs and proves the merge-neuron theorem. Section IV analyses the three crystallization strategies. Section V presents the experimental evaluation. Section VI positions the work relative to the literature. Sections VII and VIII discuss implications and conclude.

II. BACKGROUND

A. Łukasiewicz many-valued logic

In classical propositional logic, truth values are drawn from $\{0, 1\}$. Łukasiewicz logic generalizes this to the real unit interval $[0, 1]$, interpreting 0 as complete falsity and 1 as complete truth, with intermediate values representing degrees of certainty [10]. The four fundamental connectives are defined as follows.

Definition 1 (Łukasiewicz connectives). *For $x, y \in [0, 1]$:*

$$x \otimes y = \max(0, x + y - 1), \quad (1)$$

$$x \Rightarrow y = \min(1, 1 - x + y), \quad (2)$$

$$\neg x = 1 - x, \quad (3)$$

$$x \oplus y = \min(1, x + y). \quad (4)$$

The operations $\otimes$ and $\oplus$ recover Boolean conjunction and disjunction when restricted to $\{0, 1\}$; they are, however, genuinely many-valued on $(0, 1)$. For instance, $\frac{1}{2} \otimes \frac{1}{2} = 0$: two half-truths do not compose into a truth under conjunction unless their sum exceeds 1. Note that $x \oplus y = \neg(\neg x \otimes \neg y)$, so disjunction is De Morgan dual to conjunction, as in classical logic. The residuum $\Rightarrow$ is the natural implication associated with the t-norm $\otimes$ [10].

A *McNaughton function* [11] is a continuous, piecewise linear map $f: [0, 1]^n \rightarrow [0, 1]$ with integer coefficients in each linear piece. McNaughton’s theorem states that f is a McNaughton function if and only if it is expressible as a formula in infinitely-valued Łukasiewicz logic [11, 12]. All four connectives in Eqs. (1)–(4) are piecewise linear with integer coefficients, confirming that the logic is exactly characterised by this function class.

For any integer $n \geq 1$, the finite set $S_n = \{0, 1/n, 2/n, \dots, 1\}$ is closed under all four operations. These finite subdomains allow truth-table-style evaluation and make the semantics computationally tractable.

B. Łukasiewicz neural networks

A neuron with m inputs $x_1, \dots, x_m \in [0, 1]$, weights $w_1, \dots, w_m \in \mathbb{R}$, bias $b \in \mathbb{R}$, and *truncated identity activation*

$$\psi(x) = \min(1, \max(0, x)), \quad (5)$$

produces output $z = \psi(\sum_{i=1}^m w_i x_i + b)$. We extend the notation to include the bias explicitly:

$$\psi_b(z) = \psi(z + b) = \min(1, \max(0, z + b)). \quad (6)$$

This activation is piecewise linear with integer coefficients, matching the structure of McNaughton functions.

The following proposition, established by Castro and Trillas [6] and extended by Leandro [7], is the central result connecting neural computation to Łukasiewicz logic.

Proposition 2 (Neuron classification [6, 7]). *Let $\alpha = \psi_b(-x_1, \dots, -x_n, x_{n+1}, \dots, x_m)$ be a neuron with n weights equal to -1 and $p = m - n$ weights equal to $+1$.*

- 1) *If $b = -p + 1$, then α computes the conjunction $\neg x_1 \otimes \dots \otimes \neg x_n \otimes x_{n+1} \otimes \dots \otimes x_m$.*
- 2) *If $b = n$, then α computes the disjunction $\neg x_1 \oplus \dots \oplus \neg x_n \oplus x_{n+1} \oplus \dots \oplus x_m$.*

Proof sketch. For case 1 with $n = 0$: $\psi_{-p+1}(x_1 + \dots + x_m) = \max(0, x_1 + \dots + x_m - (p - 1)) = x_1 \otimes \dots \otimes x_m$, which follows by induction using $x \otimes y = \max(0, x + y - 1)$. For $n > 0$, substitute $\neg x_i = 1 - x_i$ for each negated input: the weighted sum becomes $\sum_{j=n+1}^m x_j - \sum_{i=1}^n x_i + n + b$. Setting $b = -p + 1 = -(m - n) + 1$ yields $\sum_{j=n+1}^m x_j + \sum_{i=1}^n (1 - x_i) - 1$, and ψ applied to this recovers the m -fold conjunction. Case 2 is analogous using $b = n$ and the disjunction definition. $\square$

A network in which *every* neuron satisfies Proposition 2 is a *Łukasiewicz neural network* (ŁNN). In a ŁNN, the output can be read as a Łukasiewicz formula by composing the sub-formulas computed at each neuron, propagating bottom-up through the layers. Conversely, any Łukasiewicz formula can be encoded as a ŁNN by mapping each connective to a neuron with the corresponding bias and weights $\{-1, 0, +1\}$ [13]. This bidirectional correspondence — formula $\leftrightarrow$ network — is the foundation of the symbolic extraction methodology.

C. Rewriting algebra and symbolic extraction

Proposition 2 covers neurons with at most two distinct weight values (-1 and $+1$). Real networks,

after training, may have neurons with more complex weight configurations that do not directly satisfy the proposition. The *rewriting algebra* of [7] handles this case.

a) *Rule R.*: Rule R peels one input off the neuron by introducing a binary sub-neuron with connection weight $+1$; the original bias is distributed as $b = b_0 + b_1$. The split is valid when (i) the original bias is distributed as $b = b_0 + b_1$, where b_0 is assigned to the left sub-neuron and b_1 to the right sub-neuron, and (ii) neither sub-neuron produces a constant output (i.e. neither ψ_{b_0} nor ψ_{b_1} is identically 0 or 1 on the relevant domain). Applying Rule R exhaustively to all valid bias distributions (b_0, b_1) generates a finite set of candidate two-neuron networks.

b) *λ -similarity.*: The proximity between two networks over a finite subdomain S_n is measured by

$$\lambda = \exp(-\text{MAE}(f(X), y)), \quad (7)$$

where MAE is the mean absolute error over all input vectors $X \in S_n^m$. When $\lambda = 1$, the two functions are identical on S_n . Since outputs lie in $[0, 1]$, $\text{MAE} \leq 1$ and hence $\lambda \geq e^{-1} \approx 0.368$; smaller values of λ indicate greater divergence between the two functions. A neuron is *representable* in the sense of Proposition 2 if and only if all applications of Rule R yield networks with $\lambda = 1$ relative to the original [7]. When a neuron is not representable, the candidate with the highest λ provides the best logical approximation.

c) *Extraction procedure.*: Given a crystallized ŁNN, symbolic extraction proceeds layer by layer, bottom-up. Each neuron is tested for direct representability (Proposition 2). If representable, the corresponding Łukasiewicz formula is recorded. If not, Rule R is applied exhaustively and the best- λ candidate replaces the neuron. The resulting formula is a Łukasiewicz expression in the input variables.

D. Relationship to Leandro (2009)

The ŁNN framework was established in [7]. Compared to that work, the present paper introduces the following novelties: (i) a theoretical guarantee for residual blocks (Theorem 4) in place of the neuron classification of Proposition 2; (ii) a corrected LM training procedure (Section IV-B) that avoids the numerical instability present in Leandro’s original implementation; (iii) two additional crystallization strategies—STE and Proximal—applied to both flat and residual architectures, whereas Leandro (2009) only experimented on MONK and small UCI sets; and (iv) a rigorous 5-way statistical comparison across six datasets with ten trials each.

Proposition 2 and the rewriting algebra in Sections II-B–II-C are reproduced from [7] as background; all material in Sections III–V is new.

III. RESIDUAL ŁUKASIEWICZ NETWORKS

The original ŁNN framework is restricted to shallow architectures; residual connections resolve this by introducing a structural constraint on the merge neuron that guarantees Proposition 2 without any additional training restriction.

A. The Łukasiewicz residual block

Definition 3 (Łukasiewicz residual block). *Let $F^{(k)}: [0, 1]^d \rightarrow [0, 1]^d$ be a feed-forward linear layer with real-valued weights during training at depth k . The Łukasiewicz residual block at depth k produces*

$$\mathbf{h}^{(k)} = \psi_{\mathbf{b}}\left(F^{(k)}(\mathbf{h}^{(k-1)}) + \mathbf{h}^{(k-1)}\right), \quad (8)$$

where $\mathbf{b} \in \mathbb{R}^d$ is a vector of merge biases that are trained and crystallized alongside the weights of $F^{(k)}$, and $\psi_{\mathbf{b}}$ denotes element-wise application of the scalar function ψ_{b_j} (Eq. (6)) to the j -th coordinate.

The output $\mathbf{h}_j^{(k)}$ is the result of a merge neuron that receives two inputs: $F^{(k)}(\mathbf{h}^{(k-1)})_j$ and $\mathbf{h}_j^{(k-1)}$. Both inputs arrive with weight +1 — the residual path contributes $F^{(k)}(\mathbf{h}^{(k-1)})_j$ with coefficient +1 and the shortcut contributes $\mathbf{h}_j^{(k-1)}$ with coefficient +1. The bias b_j is the only free parameter of the merge neuron.

B. Symbolic guarantee: Proposition 2 by construction

The structural observation above leads directly to the main theoretical result of this paper.

Theorem 4 (Merge neurons satisfy Proposition 2 by construction). *Let $b_j \in \mathbb{Z}$ be the (crystallized) merge bias for component j in a Łukasiewicz residual block (Definition 3). Then the merge neuron for component j satisfies Proposition 2:*

- 1) If $b_j = -1$, the merge neuron computes the conjunction $F_j^{(k)} \otimes h_j^{(k-1)}$.
- 2) If $b_j = 0$, the merge neuron computes the disjunction $F_j^{(k)} \oplus h_j^{(k-1)}$.

No constraint on the weights of $F^{(k)}$ is required.

Proof. The merge neuron for component j computes $\psi(F_j^{(k)} + h_j^{(k-1)} + b_j)$ with input weights (+1, +1). Setting $n = 0$ (zero negative weights) and $p = 2$ (two positive weights) in Proposition 2:

- Case 1 ($b_j = -p + 1 = -1$): $\psi(F_j^{(k)} + h_j^{(k-1)} - 1) = \max(0, F_j^{(k)} + h_j^{(k-1)} - 1) = F_j^{(k)} \otimes h_j^{(k-1)}$. ✓
- Case 2 ($b_j = n = 0$): $\psi(F_j^{(k)} + h_j^{(k-1)}) = \min(1, F_j^{(k)} + h_j^{(k-1)}) = F_j^{(k)} \oplus h_j^{(k-1)}$. ✓

Both cases follow from the definitions in Eqs. (1) and (4). The merge bias values $b_j \in \{-1, 0\}$ are the only integer values consistent with $n = 0, p = 2$ in Proposition 2; all other integer biases would produce constant outputs on $[0, 1]^2$ (e.g. $b_j = -2$: $\psi(x + y - 2) = 0$ for all $x, y \in [0, 1]$; $b_j = 1$: $\psi(x + y + 1) = 1$ for all $x, y \in [0, 1]$). □

Remark 5. *The proof is a direct application of Proposition 2 to the specific parameterisation $n = 0, p = 2$ imposed by the residual block structure. The contribution of Theorem 4 is the architectural observation that incorporating ResNet skip connections into ŁNNs forces exactly this parameterisation at the merge neuron, establishing the symbolic guarantee without any additional training constraint.*

Corollary 6. *In a Łukasiewicz residual network, every merge neuron is interpretable as either conjunction or disjunction, independently of how well the inner layer $F^{(k)}$ has crystallized.*

This result has a practical consequence: even when the inner layers of the residual block contain neurons that are not directly representable by Proposition 2 (requiring the rewriting algebra for approximation), the merge neurons are guaranteed to be exact. The interpretability of the network’s *skeleton* (the residual connections and merge operations) is preserved by construction.

C. Formula extraction from residual networks

Extraction from a residual ŁNN proceeds by the same bottom-up procedure described in Section II-C, with the following addition. At each residual block, the merge neuron is extracted first using Theorem 4: if the crystallized bias $b_j \in \{-1, 0\}$, the merge formula is $F_j^{(k)} \otimes h_j^{(k-1)}$ or $F_j^{(k)} \oplus h_j^{(k-1)}$, respectively. The sub-formula $F_j^{(k)}$ is then extracted from the inner layer neurons using Proposition 2 (with the rewriting algebra for non-representable cases).

As a concrete illustration, the residual ŁNN trained on the MONK-3 problem (Section V) converges to the exact formula

$$F = \neg(\text{body_shape} \oplus \text{jacket_colour}), \quad (9)$$

with the merge neuron taking $b_j = -1$ (conjunction of the shortcut and the hidden-layer output) and the

inner layer computing $body_shape \oplus jacket_colour$. The formula achieves 97.2% classification accuracy, correctly capturing the ground truth rule.

IV. CRYSTALLIZATION STRATEGIES

Proposition 2 requires all weights to be elements of $\{-1, 0, +1\}$. Gradient-based training evolves weights continuously in $\mathbb{R}$; the process of driving them to integer values is called *crystallization*. We measure crystallization quality by the *representation error*

$$\Delta(N) = \sum_k (w_k - \lfloor w_k \rfloor)^2, \quad (10)$$

where $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer. A network is crystallized when $\Delta(N) < \varepsilon$ for a small threshold ε (we use $\varepsilon = 10^{-3}$ throughout).

A. Smooth crystallization

Hard rounding destroys differentiability and prevents gradient flow. The *smooth crystallization function*

$$\Upsilon_n(w) = \text{sgn}(w) \left(\cos^n \left((1 - \{w\}) \frac{\pi}{2} \right) + \lfloor w \rfloor \right), \quad (11)$$

where $\{w\} = |w| - \lfloor |w| \rfloor$ is the fractional part, deforms the identity toward the nearest integer while remaining differentiable everywhere. The exponent n controls the strength of attraction: for $n = 2$, the function is a gentle cosine deformation; as $n \rightarrow \infty$, it converges to hard rounding.

A *progressive crystallization schedule* applies Υ_n with increasing n over the final training epochs: $\Upsilon_2 \rightarrow \Upsilon_4 \rightarrow \Upsilon_8 \rightarrow \Upsilon_{16}$. This schedule avoids premature commitment (early rounding can destroy gradient information) while ensuring that all quasi-integer weights are driven to the correct integer before the final round. At termination, weights are clamped to $[-1, +1]$ and rounded; biases are rounded but not clamped, preserving multi-input conjunctions and disjunctions.

a) *Interaction with STE*.: For the STE strategy (Section IV-C), the smooth schedule is applied to the continuous shadow weights w_c between update steps during the final training epochs: each w_c is replaced by $\Upsilon_n(w_c)$ before the hard quantization $T(\Upsilon_n(w_c))$ is computed for the next forward pass. This ensures that shadow weights enter the final rounding in a state that is already close to the target integer, reducing the gradient variance introduced by the STE approximation near the quantization boundaries.

B. Levenberg–Marquardt training

The Levenberg–Marquardt algorithm [14, 15] is a second-order method that interpolates between gradient descent and the Gauss–Newton method. Given residuals $\mathbf{e} = f_\theta(X) - y$ and Jacobian $J = \partial \mathbf{e} / \partial w$, the weight update is

$$\Delta w = -[J^\top J + \mu I]^{-1} J^\top \mathbf{e}, \quad (12)$$

where $\mu > 0$ is the Levenberg damping parameter.

a) *A critical correction*.: The original implementation in [7] used the Marquardt scaling $\mu \text{diag}(J^\top J)$ in place of μI . When weights are near zero at initialization, the diagonal entries of $J^\top J$ are also near zero; the system becomes ill-conditioned. To quantify: if $\|w\| \approx \epsilon \ll 1$, the Jacobian entries $J_{ij} = \partial e_i / \partial w_j$ are $O(\epsilon)$ because the truncated-identity activation is nearly linear near zero, giving $\text{diag}(J^\top J) \sim O(\epsilon^2)$. The Marquardt-damped system has effective regularization $(1 + \mu) \text{diag}(J^\top J) \approx \mu \epsilon^2 I$, yielding $\|\Delta w\| \approx \|\mathbf{e}\| / (\mu \epsilon^2)$. For $\mu = 0.01$, $\epsilon = 0.1$, $\|\mathbf{e}\| = O(1)$, this gives $\|\Delta w\| \sim 100$, consistent with the divergent magnitudes ($\|\Delta w\| \approx 47$) observed in practice at the typical initialization scale. The Levenberg form in Eq. (12) uses the identity matrix, guaranteeing $\|\Delta w\| \leq \|\mathbf{e}\| / \mu$ regardless of the weight magnitude. All experiments in this paper use the corrected update.

b) *Properties*.: LM is a second-order method and converges rapidly near a solution: on structured small-scale problems (the four MONK datasets), the corrected LM achieves crystallization in a mean of 6.2 iterations on MONK-3. The crystallization guarantee is, however, *probabilistic*: the algorithm converges to a continuous optimum, which may or may not lie sufficiently close to the integer lattice within the allocated iteration budget. With residual connections, the structural constraint on merge neurons (Theorem 4) reduces the number of weights that must independently crystallize, improving the empirical crystallization rate substantially (Section V).

C. Straight-Through Estimation

The straight-through estimator (STE), introduced by Bengio et al. [9], resolves the differentiability problem of weight quantization through a deliberate gradient approximation.

During the forward pass, each continuous weight $w_c \in \mathbb{R}$ is quantized to the nearest element of $\{-1, 0, +1\}$ by

$$T(w_c) = \text{sgn}(w_c) \cdot \mathbf{1}[\lfloor w_c \rfloor > \frac{1}{3}]. \quad (13)$$

The threshold $\frac{1}{3}$ implements the minimum-distortion uniform ternary partition of $[-1, +1]$: it divides the interval into three equal-width regions $(-\infty, -\frac{1}{3}) \rightarrow -1$, $[-\frac{1}{3}, +\frac{1}{3}) \rightarrow 0$, $(\frac{1}{3}, +\infty) \rightarrow +1$, which is the canonical symmetric codebook for ternary quantization with a uniform prior on $[-1, +1]$. The network computes with $T(w_c)$ but stores and updates the continuous shadow weight w_c . During the backward pass, the gradient of the loss with respect to w_c is approximated by passing it directly through T as if T were the identity:

$$\frac{\partial \mathcal{L}}{\partial w_c} \approx \frac{\partial \mathcal{L}}{\partial T(w_c)}. \quad (14)$$

The approximation is biased but has been shown to work in practice for binarized and ternarized networks [16, 17]: the continuous weights learn to position themselves in regions where T is locally constant, and the direction of the approximate gradient is sufficiently informative.

a) Crystallization guarantee.: By construction, the final integer weights are $T(w_c)$ for each parameter. Crystallization is 100% guaranteed regardless of the problem structure or dimensionality. The cost is that all n parameters are used (no sparsity), producing networks whose inner neurons may not directly satisfy Proposition 2; the rewriting algebra (Section II-C) is then required to extract a logical formula.

Applied to LukResidualNet, STE requires no algorithmic modification: inner layers carry `mode='ste'` parameters quantized by $T(\cdot)$; merge biases are treated as continuous shadow weights and rounded after training. By Theorem 4, crystallized merge biases $b_j \in \{-1, 0\}$ yield exact conjunction or disjunction regardless of inner-layer representability, providing a learnable pass-through that benefits datasets whose ground-truth logic is expressible by a shallow formula.

D. Proximal regularization

The proximal strategy trains with Adam [18] on a composite objective that jointly minimizes prediction error and drives weights toward integer values:

$$\mathcal{L} = \text{MSE} + \lambda_s \sum_k |w_k| + \lambda_a \sum_k w_k^2 (1 - w_k^2). \quad (15)$$

The first penalty, $\lambda_s \sum |w_k|$, is an ℓ_1 regularizer that encourages sparsity by driving weights toward zero. The second penalty, $\lambda_a \sum w_k^2 (1 - w_k^2)$, is the *ternary attraction potential* [19]: the function $P(w) = w^2(1 - w^2)$ vanishes exactly at $w \in \{-1, 0, +1\}$

and achieves its maximum at $|w| = 1/\sqrt{2} \approx 0.71$, creating a potential well around each integer target.

Training proceeds in two phases. In *Phase 1* (65% of the iteration budget), only the MSE term is active, allowing the network to find a continuous optimum without interference from the regularizers. The 65/35 split was selected so that Phase 1 is long enough for the network to locate a good loss basin before crystallization pressure is applied; starting the regularizers too early (e.g. 50/50) causes the ℓ_1 term to collapse weights before the network has converged, while too late a start (e.g. 80/20) leaves insufficient budget for reliable crystallization. In *Phase 2* (the remaining 35%), λ_s and λ_a are introduced gradually and then amplified by a factor of $10\times$ in the final segment, pushing all near-integer weights to their nearest integer. Crystallization is guaranteed by construction, as with STE.

a) Sparsity and failure mode.: The ℓ_1 penalty has the beneficial effect of driving many weights exactly to zero, producing sparse networks: the Heart Disease experiment (Section V-D0b) extracts a formula using only 5 of the available 22 input features. The same penalty creates a failure mode on high-dimensional dense problems: the ℓ_1 regularizer eliminates discriminative weights before the ternary attraction potential can stabilize a non-trivial solution, collapsing the network to a constant-output classifier. This failure manifests as $F_1 \approx 0$ and ± 0 standard deviation across all restarts (Section V).

Applied to LukResidualNet, the proximal objective and two-phase schedule are unchanged; Theorem 4 guarantees merge-bias crystallization independently. The residual shortcut does not mitigate the ℓ_1 collapse: Proximal-Res fails on the same five of six datasets as flat Proximal (Section V), confirming that the collapse is driven by the ℓ_1 -gradient interaction rather than by signal loss through depth.

E. Theoretical comparison

Table I summarizes the theoretical properties of the three strategies.

Every residual configuration benefits from Theorem 4: merge neurons satisfy Proposition 2 by construction, regardless of the inner-layer training dynamics. For the residual variants, the “Merge exact” column captures this architectural guarantee; the inner neurons may still require the rewriting algebra for complete formula extraction. LM with flat architecture omits the “Merge exact” column because it produces no merge neurons.

TABLE I

THEORETICAL PROPERTIES OF THE THREE CRYSTALLIZATION STRATEGIES. “CRYSTALLIZATION GUARANTEED”: ALL TRIALS ACHIEVE $\Delta(N) < \varepsilon$. “SPARSITY”: TENDENCY TO PRODUCE ZERO WEIGHTS. “EXACT PROP. 2”: EXTRACTED FORMULA SATISFIES THE PROPOSITION EXACTLY WITHOUT REWRITING-ALGEBRA APPROXIMATION.

Strategy	Arch.	Cryst. guar.	Sparsity	Merge exact
LM	Flat	Probabilistic	No	N/A
LM	Residual	Probabilistic	No	Yes
STE	Flat	Yes	No	N/A
STE	Residual	Yes	No	Yes
Proximal	Flat	Yes	Yes	N/A
Proximal	Residual	Yes	Yes	Yes

F. Training and extraction pipeline

Algorithm 1 summarises the complete workflow for any of the three strategies.

V. EXPERIMENTAL EVALUATION

A. Protocol

a) *Datasets.*: We evaluate on six UCI [20] benchmarks representing a range of problem sizes and structures (Table II).

TABLE II

DATASET CHARACTERISTICS. CLASS BALANCE IS THE FRACTION OF THE MAJORITY CLASS (TEST SET). [†]MUSHROOM TRAINING DATA AUGMENTED WITH SYNTHETIC HALF-VALUED NEGATIVE INSTANCES (EDIBLE MUSHROOMS RESCALED TO 0.5); ENRICHED BALANCE SHOWN.

Dataset	Train	Test	Features	Class balance
Mushroom [†]	9866	2466	111	65.9%
Heart Disease	243	60	22	54.1%
MONK-1	124	432	17	50.0%
MONK-2	169	432	17	67.1%
MONK-3	122	432	17	52.8%
Breast Cancer	229	57	15	70.3%

The Heart Disease dataset (Cleveland subset) uses 13 raw attributes; five continuous features are scaled to $[0, 1]$ via min-max normalization, three binary attributes are kept as-is, four categorical attributes are one-hot encoded, and one ordinal attribute (ca) is normalized to $[0, 1]$, yielding 22 input features. MONK datasets use nominal attributes encoded as one-hot binary vectors; MONK-3 includes 5% attribute noise [21].

b) *Training setup.*: For each dataset and configuration, we run 10 independent trials with different random seeds and select hyperparameters (architecture, learning rate, and regularization strengths) by grid search on a validation split, reporting test set results. The five configurations are: LM with residual

Algorithm 1 LNN training and symbolic extraction

Require: Dataset (X, y) ; strategy $s \in \{\text{LM-Res, STE, STE-Res, Proximal, Prx-Res}\}$; budget T ; threshold ε

Ensure: Łukasiewicz formula F or best- λ approximation

- 1: Initialise $w \leftarrow$ small random values; $b \leftarrow 0$
- 2: **for** $t = 1$ **to** T **do**
- 3: **Train** with strategy s : LM-Res via Eq. (12); STE/STE-Res via Eqs. (13),(14); Proximal/Prx-Res via Eq. (15) with two-phase schedule
- 4: **if** $t > 0.85T$ **then**
- 5: Apply smooth schedule $\Upsilon_2 \rightarrow \Upsilon_{16}$ to shadow weights w_c
- 6: **end if**
- 7: **end for**
- 8: Clamp $w_c \leftarrow \text{clip}(w_c, -1, +1)$; round all w_c and b to nearest integer
- 9: Compute $\Delta(N)$ (Eq. (10)); crystallized iff $\Delta(N) < \varepsilon$
- 10: **Symbolic extraction** (bottom-up):
- 11: **for** each neuron α in topological order **do**
- 12: **if** α is a merge neuron with $b_\alpha \in \{-1, 0\}$ **then**
- 13: Apply Theorem 4 (exact conjunction / disjunction)
- 14: **else if** α satisfies Proposition 2 **then**
- 15: Record Łukasiewicz formula for α
- 16: **else**
- 17: Apply Rule R exhaustively; keep candidate with highest λ
- 18: **end if**
- 19: **end for**
- 20: **return** Composed formula F over input variables

connections (LM-Res), STE with flat architecture (STE), STE with residual connections (STE-Res), Proximal with flat architecture (Proximal), and Proximal with residual connections (Prx-Res). LM is evaluated only with residual architecture; the flat LM is the original method of [7] (its corrected form was already evaluated in the prior conference version of this work). All experiments use the smooth crystallization schedule of Section IV-A. Statistical comparisons use the Wilcoxon signed-rank test ($n = 10$, $\alpha = 0.05$). On synthetic truth tables, LM achieves exact recovery when crystallization succeeds ($F_1 = 0.720$ on a two-clause formula, 5 of 10 trials); Proximal collapses to constant output on the same formula (all 10 trials), confirming the failure mode of Section IV-D.

c) *Evaluation metrics.*: We report mean accuracy and F_1 score over 10 trials, with standard deviations, together with crystallization rate (fraction of trials achieving $\Delta(N) < 10^{-3}$). Training time (wall clock, seconds) is reported separately.

B. Crystallization reliability on real datasets

Proximal and Prx-Res are included as structural negative controls: their collapse is reproducible across all hyperparameter configurations and theoretically explained in Section IV-D. Tables III and IV report accuracy, F_1 , and crystallization rates for all five configurations across six datasets. STE, STE-Res, Proximal, and Prx-Res achieve 10/10 crystallization on every dataset by construction. LM-Res crystallizes reliably only when a near-integer optimum is reachable within the iteration budget: 10/10 on MONK-3, 9/10 on Heart Disease, 7/10 on MONK-2, 6/10 on Mushroom, 5/10 on MONK-1, and 0/10 on Breast Cancer.

The Heart Disease result (9/10 for LM-Res) is noteworthy: the flat LM without residual connections achieves 0/10 crystallization on the same problem [7]. The merge-neuron constraint of Theorem 4 reduces the effective number of weights that must independently crystallize, making the problem more tractable for the second-order solver. This improvement is architecture-specific to LM; the guaranteed-crystallization methods (STE, Proximal, and their residual variants) do not benefit from this reduction since crystallization is enforced by design in all trials.

C. Classification accuracy and F_1

No single configuration dominates across all datasets. STE-Res achieves the highest accuracy on MONK-1 (0.674, +12.4 pp over flat STE) and MONK-3 (0.717, +3.6 pp over STE), narrowly edging out LM-Res (0.714) on MONK-3. STE leads on MONK-2 (+4.8 pp over STE-Res) and Heart Disease. LM-Res leads on Mushroom (0.668).

The F_1 picture is clearer: STE-Res achieves the highest mean F_1 on four of six datasets (MONK-1: 0.648, MONK-2: 0.455, MONK-3: 0.716, Mushroom: 0.299), with the differences on MONK-2 and MONK-3 statistically significant (Wilcoxon $p = 0.020$ and $p = 0.004$ respectively vs flat STE; all four STE-Res vs Prx-Res gaps are significant at $p \leq 0.016$). The F_1 improvement on Heart Disease is an exception: STE-Res (0.272) lags behind STE (0.482), suggesting that the residual connections hinder performance on this small, low-dimensional problem. The accuracy differences between STE and

STE-Res are not statistically significant on any individual dataset (all accuracy Wilcoxon $p > 0.05$), but the consistent direction of the F_1 advantage on three of four structured MONK datasets supports the hypothesis that residual connections benefit problems with underlying logical structure.

The degenerate Proximal solutions on five of six datasets (accuracy equals majority-class rate with ± 0.000 standard deviation, $F_1 = 0$) confirm the failure mode as structural: grid search was conducted over a wide range of λ_s and λ_a values, and collapse persists regardless of architecture. Prx-Res shows identical degenerate behaviour to flat Proximal on all five affected datasets, confirming that the residual shortcut does not mitigate the ℓ_1 collapse. The single non-degenerate Proximal result is flat Proximal on MONK-1 ($F_1 = 0.147$); Prx-Res on MONK-1 collapses completely.

D. Formula quality and interpretability

Across all experiments, the only fully representable formula in the sense of Proposition 2 — a formula extractable without any rewriting-algebra approximation — emerges from LM-Res on MONK-3.

a) *MONK-3 exact formula.*: In trial 1, LM-Res converges in 6 iterations to the fully crystallized network with accuracy 97.2%. The extracted formula is:

$$F = \neg(\text{body_shape} \oplus \text{jacket_colour}), \quad (16)$$

which classifies all but 2.8% of the test set (the erroneous labels introduced by 5% noise). All neurons satisfy Proposition 2 exactly: the inner layer computes $\text{body_shape} \oplus \text{jacket_colour}$ ($b = 0$, case 2), the merge neuron computes $y \otimes y$ ($b_j = -1$, case 1), and on binary inputs $y \otimes y = y$, so the output negation closes the formula. STE and Proximal also crystallize on MONK-3 (10/10) but require the rewriting algebra for inner neurons.

b) *Heart Disease formula extraction.*: We highlight a Proximal trial that illustrates the framework's capability for clinically interpretable rule extraction. This trial was conducted with manually selected hyperparameters outside the grid search; the systematic grid search (Section V-C) finds that Proximal collapses to a majority-class solution on Heart Disease across all tested hyperparameter combinations. The trial below therefore illustrates a *best-case capability* of the Proximal strategy rather than its typical behaviour on this dataset. After 321 iterations, the trial produces a maximally sparse crystallized network on the Heart Disease dataset. Four of the six first-layer

TABLE III

MEAN ACCURACY ($\pm$ STD, 10 TRIALS) AND CRYSTALLIZATION RATE FOR ALL FIVE CONFIGURATIONS. *: WILCOXON $p < 0.05$ (STE-RES $>$ PRX-RES). BEST ACCURACY PER DATASET IN **BOLD**. †: LM-RES ACHIEVED 0/10 CRYSTALLIZATION ON BREAST CANCER; ACCURACY IS FROM THE UNCRYSTALLIZED CONTINUOUS MODEL (NO FORMULA EXTRACTED).

Dataset	Accuracy (mean $\pm$ std)					Crystallization rate				
	LM-Res	STE	STE-Res	Proximal	Prx-Res	LM-Res	STE	STE-Res	Proximal	Prx-Res
Mushroom	.668 $\pm$.028	.665 $\pm$.040	.626 $\pm$.102	.660 $\pm$.000	.660 $\pm$.000	6/10	10/10	10/10	10/10	10/10
Heart Disease	.592 $\pm$.122	.607 $\pm$.101	.558 $\pm$.088	.600 $\pm$.000	.600 $\pm$.000	9/10	10/10	10/10	10/10	10/10
MONK-1*	.515 $\pm$.296	.600 $\pm$.125	.674 $\pm$.118	.558 $\pm$.125	.500 $\pm$.000	5/10	10/10	10/10	10/10	10/10
MONK-2	.631 $\pm$.243	.704 $\pm$.070	.656 $\pm$.145	.671 $\pm$.000	.671 $\pm$.000	7/10	10/10	10/10	10/10	10/10
MONK-3*	.714 $\pm$.212	.681 $\pm$.158	.717 $\pm$.149	.472 $\pm$.000	.472 $\pm$.000	10/10	10/10	10/10	10/10	10/10
Breast Cancer	.632 $\pm$.000[†]	.572 $\pm$.120	.605 $\pm$.087	.632 $\pm$.000	.632 $\pm$.000	0/10	10/10	10/10	10/10	10/10

*MONK-1: $p(\text{STE-Res} > \text{Prx-Res}) = 0.008$; MONK-3: $p(\text{STE-Res} > \text{Prx-Res}) = 0.004$ (Wilcoxon, $n = 10$).

TABLE IV

MEAN F_1 SCORE ($\pm$ STD, 10 TRIALS). *: WILCOXON $p < 0.05$. BEST F_1 PER DATASET IN **BOLD**.

Dataset	LM-Res	STE	STE-Res	Proximal	Prx-Res
Mushroom	.103 $\pm$.223	.229 $\pm$.316	.299 $\pm$.298	.000 $\pm$.000	.000 $\pm$.000
Heart Disease	.462 $\pm$.325	.482 $\pm$.337	.272 $\pm$.330	.000 $\pm$.000	.000 $\pm$.000
MONK-1*	.513 $\pm$.263	.481 $\pm$.337	.648 $\pm$.240	.147 $\pm$.311	.000 $\pm$.000
MONK-2*	.366 $\pm$.347	.179 $\pm$.286	.455 $\pm$.267	.000 $\pm$.000	.000 $\pm$.000
MONK-3*	.603 $\pm$.425	.612 $\pm$.331	.716 $\pm$.259	.000 $\pm$.000	.000 $\pm$.000
Breast Cancer	.000 $\pm$.000	.109 $\pm$.230	.137 $\pm$.196	.000 $\pm$.000	.000 $\pm$.000

*MONK-1: $p(\text{STE-Res} > \text{Prx-Res})_{F_1} = 0.004$; MONK-2: $p(\text{STE} < \text{STE-Res})_{F_1} = 0.020$, $p(\text{STE-Res} > \text{Prx-Res})_{F_1} = 0.004$; MONK-3: $p(\text{STE-Res} > \text{Prx-Res})_{F_1} = 0.004$; Mushroom: $p(\text{STE-Res} > \text{Prx-Res})_{F_1} = 0.016$.

neurons carry only zero weights and are discarded by the symbolic extractor. The two active neurons are:

$$\begin{aligned} n_1 &= \psi_{+1}(\neg \text{trestbps}, \neg \text{oldpeak}, \neg \text{ca}) \\ &= \neg \text{trestbps} \otimes \neg \text{oldpeak} \otimes \neg \text{ca}, \end{aligned} \quad (17)$$

$$\begin{aligned} n_5 &= \psi_0(+\text{thalach}, \neg \text{ca}, \neg \text{cp}=4) \\ &= \text{thalach} \otimes \neg \text{ca} \otimes \neg (\text{cp}=4). \end{aligned} \quad (18)$$

where the bias conditions $b = -p+1 = +1$ in n_1 and $b = n = 0$ in n_5 both satisfy Proposition 2 (conjunctions with three inputs). The output layer combines n_1 and n_5 by: $i_1 = \psi_{+1}(\neg n_1, \neg n_5) = \neg n_1 \otimes \neg n_5$. Substituting yields the complete formula:

$$\begin{aligned} \text{DISEASE} &= \neg(\neg \text{trestbps} \otimes \neg \text{oldpeak} \otimes \neg \text{ca}) \\ &\quad \otimes \neg(\text{thalach} \otimes \neg \text{ca} \otimes \neg (\text{cp}=4)) \end{aligned} \quad (19)$$

(DISEASE = 1 indicates the presence of coronary artery disease).

c) *Numerical evaluation.*: Table V evaluates Eq. (19) for two representative test patients using the Łukasiewicz conjunction $a \otimes b = \max(0, a + b - 1)$. Patient A has complete vessel occlusion ($\text{ca}=1.00$), which zeroes both n_1 and n_5 , driving the output to 1.00 (disease). Patient B presents with no stenosis, normal blood pressure, and good exercise capacity;

the output is 0.00 (healthy). A threshold of 0.5 separates the two correctly. The formula can be verified step by step by any reader who knows the single operator definition $a \otimes b = \max(0, a + b - 1)$.

TABLE V
EVALUATION TRACE FOR EQ. (19) ON TWO TEST PATIENTS
(FEATURES NORMALISED TO [0, 1]).

	Patient A (disease)	Patient B (healthy)
ca	1.00	0.00
trestbps	0.64	0.27
oldpeak	0.42	0.00
thalach	0.29	0.79
cp=4 indicator	1	0
$\neg \text{ca}$	0.00	1.00
$\neg \text{trestbps}$	0.36	0.73
$\neg \text{oldpeak}$	0.58	1.00
$\neg (\text{cp}=4)$	0.00	1.00
$n_1 = \neg \text{t} \otimes \neg \text{o} \otimes \neg \text{ca}$	0.00	0.73
$n_5 = \text{tal} \otimes \neg \text{ca} \otimes \neg (\text{cp}=4)$	0.00	0.79
$\neg n_1$	1.00	0.27
$\neg n_5$	1.00	0.21
DISEASE = $\neg n_1 \otimes \neg n_5$	1.00	0.00

t=trestbps, o=oldpeak, tal=thalach. Values are illustrative from the test set.

Only 5 of the 22 input features received non-

zero weights (Table VI). Every selected feature is an established predictor of coronary artery disease in the cardiology literature [1, 22]. Most striking is the selection consistency: ca (the number of major vessels with $> 50\%$ stenosis) is the only feature selected in every one of the 10 crystallized Proximal trials, regardless of random initialization. We note that ca is universally recognized as the dominant predictor in the Cleveland dataset [22]; the result is therefore not a clinical discovery but a *validation* that the Proximal extractor reliably reproduces established medical knowledge, confirming that the ℓ_1 regularizer does not spuriously suppress clinically established predictors under initialization variance.

TABLE VI
FEATURES SELECTED IN THE HEART DISEASE FORMULA (EQ. 19), WITH CLINICAL INTERPRETATION. “ESTABLISHED”: RECOGNIZED PREDICTOR OF CORONARY ARTERY DISEASE [22].

Feature	Clinical meaning
ca	Number of major vessels with $> 50\%$ stenosis
$trestbpts$	Resting systolic blood pressure (mmHg)
$oldpeak$	ST-segment depression at exercise
$thalach$	Maximum heart rate achieved (bpm)
$cp=4$	Asymptomatic chest pain presentation

Evaluated on the held-out test set (61 patients: 37 healthy, 24 with disease), formula (19) achieves accuracy 85.0%, sensitivity 87.5%, specificity 83.3%, and $F_1 = 0.824$, identifying 21 of 24 disease cases correctly. The formula can be verified analytically by any clinician: it classifies a patient as diseased when neither the triple of low-risk indicators (normal resting blood pressure, no ST depression, no blocked vessels) nor the triple of adequate cardiac reserve indicators (high maximum heart rate, no blocked vessels, symptomatic chest pain) are simultaneously satisfied.

E. Training efficiency

Training times are dominated by architecture-strategy interactions. LM-Res is the fastest on MONK-3 (1.25 s, mean 6.2 iterations) owing to second-order convergence near the integer lattice, but is the slowest on Breast Cancer (15.4 s): the Jacobian inversion has $O(p^2)$ cost per iteration, and with 102 mean iterations on a 20-feature dataset, the per-iteration cost dominates. On Mushroom (111 features), LM-Res takes 10.2 s while Prx-Res takes 4.5 s, reflecting the quadratic-versus-linear cost advantage of first-order methods on high-dimensional problems.

STE-Res is substantially slower than flat STE: 368.8 s versus 88.7 s on Mushroom (the residual

architecture requires 1114 vs 461 mean iterations), and 2.79 s versus 1.00 s on Heart Disease (459 vs 175 iterations). STE-Res converges fastest of all on MONK-3 (0.21 s, 154 iterations), where the residual pass-through helps reach a good solution quickly.

Proximal and Prx-Res are the fastest configurations on most datasets, because Phase 1 (pure MSE minimization) terminates aggressively and the two-phase schedule limits total iterations. This speed advantage comes at a high cost in solution quality: Proximal and Prx-Res produce degenerate solutions on five of six datasets (Section V-C), so the fast convergence reflects early collapse rather than efficient learning.

VI. RELATED WORK

A. Rule extraction from neural networks

Extracting symbolic rules from trained neural networks has been studied since the late 1980s. Gallant [3] proposed connectionist expert systems in which threshold units encode conjunctions; the extracted rules are Boolean. KBANN [5] injects domain knowledge into a network and refines it through training, then extracts updated rules via a search over activation patterns. Towell and Shavlik [4] systematized this approach with a weight-based extraction heuristic. All these methods extract rules *post-hoc*: the network is trained as a continuous model and then approximated by a symbolic rule. The approximation introduces an irreducible fidelity gap.

The ŁNN approach is qualitatively different: the symbolic formula is not an approximation to the network but an exact reading of it. When all neurons satisfy Proposition 2, the network *is* the formula. When the rewriting algebra is needed, the approximation is bounded and measurable by the λ -similarity metric rather than being assessed informally.

B. Fuzzy and many-valued neural systems

ANFIS [23] and related fuzzy neural architectures learn membership functions and combine them with T-norm rules, producing interpretable models that resemble ŁNNs in spirit. The key difference is that ANFIS membership functions are learned as continuous parameters without a crystallization step; no exact correspondence between neurons and logical connectives is established or enforced. Amato et al. [13] proved that neurons with rational weights can represent rational Łukasiewicz functions, extending the scope of the Castro–Trillas result, but did not provide a training algorithm for integer weights. The present work provides such an algorithm (in three variants) and the corresponding theoretical guarantees.

TABLE VII
MEAN WALL-CLOCK TRAINING TIME (SECONDS, 10 TRIALS). FASTEST CONFIGURATION PER DATASET IN **BOLD**. MEAN ITERATION COUNTS ARE GIVEN IN PARENTHESES.

Dataset	LM-Res	STE	STE-Res	Proximal	Prx-Res
Mushroom	10.24 (255)	88.71 (461)	368.80 (1114)	6.88 (44)	4.54 (24)
Heart Disease	2.25 (96)	1.00 (175)	2.79 (459)	0.23 (39)	0.16 (29)
MONK-1	6.34 (143)	0.39 (306)	0.53 (377)	0.57 (217)	0.44 (83)
MONK-2	4.77 (135)	3.30 (482)	2.46 (372)	4.37 (568)	1.85 (108)
MONK-3	1.25 (6)	0.30 (253)	0.21 (154)	0.35 (49)	0.37 (43)
Breast Cancer	15.42 (102)	3.29 (561)	4.23 (719)	0.90 (154)	0.93 (160)

A contemporaneous line of work applies the Łukasiewicz T-norm in a different architectural setting. Nguy and Wasilewski [24] study Differentiable Logic Gate Networks (DLNs) in which each computational unit learns a probability distribution over the 16 Boolean operations via softmax; the Łukasiewicz T-norm $T_L(a, b) = \max\{0, a + b - 1\}$ replaces the Menger T-norm as the fuzzy relaxation. Post-training, each unit is snapped to its most probable gate, converting the DLN into a Binary Logic Gate Network (LGN) — a process analogous to crystallization. To address vanishing gradients in deeper DLNs, they employ a residual initialization scheme that biases units toward pass-through gates, an engineering heuristic that parallels our use of skip connections but is not backed by a structural theorem. Three distinctions separate the two approaches: (i) DLNs select *which* operation to apply at each node, whereas ŁNNs fix the Łukasiewicz semantics and crystallize *connection weights*; (ii) DLN discretization yields logic-gate circuits without an explicit logical formula, while ŁNN crystallization directly exposes a readable Łukasiewicz formula; (iii) the residual guarantee of Theorem 4 is a formal structural property, whereas DLN residual initialization is a training heuristic motivated by gradient flow.

C. Weight quantization

The STE used in Section IV-C was developed in the context of binarized neural networks [9, 16], where weights are restricted to $\{-1, +1\}$ for hardware efficiency. Ternary weight networks [17] extend this to $\{-1, 0, +1\}$, matching the target domain of crystallization. The objective of quantization in these works is computational efficiency, not logical interpretability: the quantized weights are not interpreted as logical connectives, and no formula extraction procedure is defined. The proximal objective of Section IV-D is superficially related to ℓ_1 -regularized training, but the ternary attraction term $P(w)$ is specific to the discrete target set $\{-1, 0, +1\}$ and

is designed to ensure that crystallization produces logically meaningful weights.

D. Neuro-symbolic AI

The broader neuro-symbolic AI programme [25, 26] seeks to combine neural learning with symbolic reasoning. Riegel et al. [28] introduced Logical Neural Networks (IBM LNNs), where every neuron represents a weighted real-valued logical operator; unlike ŁNNs, IBM LNN weights remain real-valued, no crystallization is defined, and no neuron-to-connective correspondence is formally established. Serafini and d’Avila Garcez [29] proposed Logic Tensor Networks (LTNs), which ground first-order formulae in neural networks via real-valued Łukasiewicz t-norm satisfaction; LTN weights remain continuous and no discrete formula is extracted. Evans and Grefenstette [30] learn first-order logic clauses by differentiating through a discrete rule space rather than a neural architecture. Manhaeve et al. [31] combine neural networks with probabilistic logic programming; their uncertainty model (probability over logical facts) complements rather than competes with Łukasiewicz degrees of truth. Unlike all of these, an ŁNN after crystallization *is* the formula: network weights constitute the symbolic representation directly, and Theorem 4 provides a structural guarantee unavailable in any of the above frameworks.

VII. DISCUSSION

A. Interpretability by construction versus post-hoc

The distinction between interpretability by construction and post-hoc explanations matters most precisely when it is hardest to verify. A post-hoc explanation is a separate model trained to approximate the decisions of the original model; its fidelity to the original is measurable but never guaranteed to be 1.0 [2]. In the ŁNN framework, when a network is fully crystallized and every neuron satisfies Proposition 2, the formula *is* the model — there is no approximation

step, and the verification question reduces to algebraic manipulation of the formula rather than statistical estimation. Theorem 4 extends this guarantee to the residual merge neurons unconditionally.

This property is particularly significant in safety-critical applications. A clinical decision rule that can be written as a Łukasiewicz formula can be audited by domain experts working from the formula alone, without reference to the training data or the training procedure. The step-by-step evaluation in Table V illustrates this: a reader who knows only $a \otimes b = \max(0, a + b - 1)$ can verify the classification of any patient independently.

B. Architecture $\times$ strategy interaction

The five-way comparison reveals that the benefit of residual connections is strategy-dependent. For STE, residual connections consistently improve F_1 on structured datasets (MONK-1, MONK-2, MONK-3), where the underlying rule is logically compact. The improvement is plausibly explained by the learnable pass-through property: STE-Res can represent shallow functions by setting inner weights to zero, while flat STE forces every activation through all layers. For Proximal, residual connections provide no benefit: the ℓ_1 collapse dominates on the same datasets regardless of architecture. This asymmetry suggests that the value of residual connections in ŁNNs depends critically on whether the crystallization strategy can effectively utilise the pass-through path — a path that Proximal’s regularization pressure systematically destroys.

C. Comparison with DLN_L (Nguy and Wasilewski, 2025)

Nguy and Wasilewski [24] report accuracy on MONK-1/2/3 and Breast Cancer using their Łukasiewicz-based DLN variants (100 runs each). Table VIII confronts those figures with our five configurations (10 trials each).

TABLE VIII
TEST ACCURACY (%) ON OVERLAPPING DATASETS. DLN_L , DLN_{L^*} FROM NGUY AND WASILEWSKI [24] (100 RUNS); OUR RESULTS FROM 10 INDEPENDENT TRIALS ($\bar{x} \pm \sigma$). BEST RESULT PER ROW IN **BOLD**.

Dataset	DLN_L	DLN_{L^*}	LM-Res	STE	STE-Res
MONK-1	100.0	100.0	51.5	60.0	67.4
MONK-2	56.9	61.6	63.1	70.4	65.6
MONK-3	94.0	78.2	71.4	68.1	71.7
Breast Cancer	70.0	74.3	63.2	57.2	60.5

DLNs lead on MONK-1/3 (100%/94.0% vs. our 67.4%/71.7%), where compact DNF formulas over binary features favour gate-selection directly. ŁNNs lead on MONK-2 (STE 70.4% vs. DLN_L 56.9%), where a parity-like concept is difficult to express as a small gate circuit. On Breast Cancer, DLN_{L^*} (74.3%) outperforms STE-Res (60.5%), reflecting DLNs’ tolerance for continuous features that need not converge to ternary weights. The two frameworks occupy complementary niches: DLNs are more accurate when the target is a Boolean formula over binary features; ŁNNs provide a richer symbolic output — a readable Łukasiewicz formula with a formal structural guarantee — when interpretability by construction is the primary goal.

D. Limitations

Several limitations of the current framework should be acknowledged. First, the LM solver has quadratic complexity in the number of parameters ($O(p^2)$ for the Jacobian inversion), making it impractical for very large networks; the Mushroom experiment (111 features) already required mini-batch Jacobian approximation [32]. Second, the Proximal failure mode — collapse to constant-output solutions — is structural and is not fully resolved by hyperparameter tuning or by incorporating residual connections. Feature pre-selection before Proximal training (using, for example, an initial STE run) is a practical mitigation, but it adds a training step. Third, the expressive power of ŁNNs is bounded by the class of McNaughton functions [10, 11]; it is not known whether every classifiable decision boundary can be approximated to arbitrary accuracy by a ŁNN of bounded depth, and this question is open. Fourth, this paper operates exclusively in the propositional Łukasiewicz framework; whether the residual block construction extends to the first-order case (where quantifiers introduce cross-input dependencies) is an open question.

VIII. CONCLUSION

The Łukasiewicz neural network framework, established in [7], rests on an exact correspondence between neurons with integer weights and logical connectives. This paper has extended the framework along two axes.

On the theoretical side, we proved that residual connections can be incorporated into ŁNNs with the symbolic guarantee preserved *by construction*: merge neurons in a Łukasiewicz residual block always satisfy Proposition 2, regardless of the training

procedure applied to the inner layers. On the practical side, we analysed three crystallization strategies (corrected LM, STE, and proximal regularization), each evaluated in both flat and residual architectures, and characterised the conditions under which each succeeds or fails.

The five-way experimental evaluation across six benchmarks yields three principal findings. First, *STE with residual connections* provides the best aggregate discriminative quality: it achieves the highest mean F_1 on four of six datasets, with significant improvements over Proximal-Res on MONK-1, MONK-2, and MONK-3. The improvement over flat STE is consistent in direction (MONK-1, MONK-2, MONK-3) but not individually statistically significant, pointing to a positive effect of the learnable pass-through in residual blocks. Second, *LM with residual connections* remains the most accurate method on Mushroom and is the fastest on structured problems (6.2 mean iterations on MONK-3), recovering exact logical formulas when crystallization succeeds. Third, *proximal regularization collapses* on five of six datasets regardless of whether residual connections are used, confirming that the failure mode is intrinsic to the ℓ_1 -dominated objective rather than to the architecture.

The Heart Disease formula (19), with five clinically validated variables and a step-by-step verifiable structure, demonstrates that the framework can produce transparent diagnostic rules from real medical data. Together, these results establish Łukasiewicz neural networks as a mature framework for interpretable neuro-symbolic learning, with a principled catalogue of integer-weight training strategies and a rigorous extension to deeper, residual architectures. Open directions include: whether a universal approximation theorem holds for McNaughton functions under bounded-depth ŁNNs; whether residuated-lattice logics beyond Łukasiewicz admit analogous neural encodings; and how residual-block count affects inner-layer crystallization dynamics.

DATA AVAILABILITY STATEMENT

All datasets used in this study are publicly available from the UCI Machine Learning Repository [20] and the Cleveland Heart Disease dataset [22]. Experimental code (including all five training configurations, tuning scripts, and benchmark runner) is available at <https://github.com/CarlosMelroLeandro/luknn-extended>.

REFERENCES

- [1] C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” *Nature Machine Intelligence*, vol. 1, pp. 206–215, 2019.
- [2] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, “A survey of methods for explaining black box models,” *ACM Computing Surveys*, vol. 51, no. 5, pp. 93:1–93:42, 2018.
- [3] S. I. Gallant, “Connectionist expert systems,” *Communications of the ACM*, vol. 31, pp. 152–169, 1988.
- [4] G. G. Towell and J. W. Shavlik, “Extracting refined rules from knowledge-based neural networks,” *Machine Learning*, vol. 13, pp. 71–101, 1993.
- [5] —, “Knowledge-based artificial neural networks,” *Artificial Intelligence*, vol. 70, pp. 119–165, 1994.
- [6] J. Castro and E. Trillas, “The logic of neural networks,” *Mathware and Soft Computing*, vol. 5, pp. 23–27, 1998.
- [7] C. Leandro, “Symbolic knowledge extraction using Łukasiewicz logics,” in *Algorithmic Learning Theory (ALT 2009)*, ser. Lecture Notes in Artificial Intelligence, vol. 5809. Springer, 2009, pp. 477–491, arXiv:1604.03099.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [9] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” in *arXiv preprint arXiv:1308.3432*, 2013.
- [10] P. Hájek, *Metamathematics of Fuzzy Logic*. Dordrecht: Kluwer Academic Publishers, 1998.
- [11] R. McNaughton, “A theorem about infinite-valued sentential logic,” *Journal of Symbolic Logic*, vol. 16, no. 1, pp. 1–13, 1951.
- [12] D. Mundici, “Interpretation of AF c^* -algebras in Łukasiewicz sentential calculus,” *Journal of Functional Analysis*, vol. 65, no. 1, pp. 15–63, 1986.
- [13] P. Amato, A. Di Nola, and B. Gerla, “Neural networks and rational Łukasiewicz logic,” *IEEE Transactions on Fuzzy Systems*, vol. 10, no. 5, pp. 506–510, 2002.
- [14] K. Levenberg, “A method for the solution of certain non-linear problems in least squares,” *Quarterly of Applied Mathematics*, vol. 2, no. 2,

- pp. 164–168, 1944.
- [15] D. W. Marquardt, “An algorithm for least-squares estimation of nonlinear parameters,” *Journal of the Society for Industrial and Applied Mathematics*, vol. 11, no. 2, pp. 431–441, 1963.
 - [16] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, “BinaryNet: Training deep neural networks with weights and activations constrained to +1 or −1,” in *arXiv preprint arXiv:1602.02830*, 2016.
 - [17] F. Li, B. Zhang, and B. Liu, “Ternary weight networks,” *arXiv preprint arXiv:1605.04711*, 2016.
 - [18] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.
 - [19] N. Parikh and S. Boyd, “Proximal algorithms,” *Foundations and Trends in Optimization*, vol. 1, no. 3, pp. 127–239, 2014.
 - [20] D. Dua and C. Graff, “UCI machine learning repository,” <http://archive.ics.uci.edu/ml>, 2019.
 - [21] S. Thrun *et al.*, “The MONK’s problems: A performance comparison of different learning algorithms,” Carnegie Mellon University, Tech. Rep. CMU-CS-91-197, 1991.
 - [22] R. Detrano, A. Janosi, W. Steinbrunn *et al.*, “International application of a new probability algorithm for the diagnosis of coronary artery disease,” *American Journal of Cardiology*, vol. 64, pp. 304–310, 1989.
 - [23] J.-S. R. Jang, “ANFIS: Adaptive-network-based fuzzy inference system,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 23, no. 3, pp. 665–685, 1993.
 - [24] C. D. Nguy and P. Wasilewski, “Deep differentiable logic gate networks based on fuzzy Łukasiewicz t-norm,” in *Proceedings of the 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*, ser. ACSIS, vol. 43, 2025, pp. 219–230.
 - [25] A. d’Avila Garcez, M. Gori, L. C. Lamb, L. Serafini, M. Spranger, and S. N. Tran, “Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning,” *Journal of Applied Logics*, vol. 6, no. 4, pp. 611–632, 2019.
 - [26] A. d’Avila Garcez and L. C. Lamb, “Neurosymbolic AI: The 3rd wave,” *Artificial Intelligence Review*, vol. 56, pp. 12 387–12 406, 2023.
 - [27] P. Hitzler, S. Hölldobler, and A. K. Seda, “Logic programs and connectionist networks,” *Journal of Applied Logic*, vol. 2, no. 3, pp. 245–272, 2004.
 - [28] R. Riegel, A. Gray, F. Rossi, N. Mohamed, J.-A. Feng, J. Palmer, D. Bouneffouf, I. Madan, and S. Trivedi, “Logical neural networks,” *arXiv preprint arXiv:2006.13155*, 2020.
 - [29] L. Serafini and A. d’Avila Garcez, “Logic tensor networks: Deep learning and logical reasoning from data and knowledge,” in *NeSy 2016 – 11th International Workshop on Neural-Symbolic Learning and Reasoning*, 2016.
 - [30] R. Evans and E. Grefenstette, “Learning explanatory rules from noisy data,” *Journal of Artificial Intelligence Research*, vol. 61, pp. 1–64, 2018.
 - [31] R. Manhaeve, S. Dumancic, A. Kimmig, T. De-meester, and L. De Raedt, “DeepProbLog: Neural probabilistic logic programming,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
 - [32] R. Battiti, “First- and second-order methods for learning: Between steepest descent and newton’s method,” *Neural Computation*, vol. 4, no. 2, pp. 141–166, 1992.